

Ai-Service-Desk

Technical Documentation

Date: July 24, 2026, Version: 1.0

Revision History

Version	Date	Author	Description
1.0	24-Jul-2026	VASANTHARAJ N	Initial draft

Table of Contents

1. Introduction	1
1.1 Purpose	1
1.2 Scope	1
1.3 Intended Audience	1
1.4 References	1
1.5 Acronyms and Definitions	1
2. System Overview	2
2.1 High-Level Architecture	2
2.2 System flow - Key Components	2
2.3 User Roles and Access Levels	2
3. Functional Requirements	3
3.1 Module/Feature Name	3
3.2 Inputs/Outputs	3
3.3 Business Rules	3
3.4 Error Handling	3
4. Non-Functional Requirements	4
4.1 Performance	4
4.2 Security	4
4.3 Availability	4
4.4 Maintainability	4
5. Data Model / Database Design	5
5.1 Entity-Relationship Diagram	5
5.2 Tables and Fields	5
5.3 Indexes and Keys.....	5
5.4 Sample Data	5
6. Integration Points.....	6
6.1 External Systems	6
6.2 APIs Used.....	6
6.3 Data Flow Diagram	6

6.4 Authentication/Authorization	6
7. Test Strategy	7
7.1 Test Scenarios	7
7.2 Expected Results	7
7.3 Traceability to Requirements	7
8. Deployment & Configuration	8
8.1 Environments	8
8.2 Setup Steps	8
8.3 Configuration Files and Parameters	8
9. Appendix	9
Screenshots	9
References	9

1. Introduction

1.1 Purpose

This document describes the design and implementation of AI Service Desk, a support-ticket management application that uses a generative AI model to draft ticket summaries and suggested agent responses. It aims to give engineers, QA, and operations staff a single reference for the system's architecture, data model, APIs, and deployment process.

1.2 Scope

In scope: the FastAPI backend, the React frontend, the PostgreSQL data model, the AWS Bedrock integration used for AI ticket summarization, and the Docker-based deployment setup. Out of scope: end-user training material, AWS account provisioning, and infrastructure-as-code for cloud hosting, none of which are part of the current repository.

1.3 Intended Audience

Backend and frontend developers extending the application, QA engineers writing test plans, DevOps engineers deploying and operating the service, and product owners evaluating current functionality and gaps.

1.4 References

Project repository: ai-service-desk-main. Key frameworks: FastAPI, SQLAlchemy 2.0, Alembic, React 19, Vite. AI provider: Amazon Bedrock Converse API. See the References section (9) for links.

1.5 Acronyms and Definitions

Key acronyms and terms used in this document:

Term	Definition
API	Application Programming Interface
ORM	Object-Relational Mapping (SQLAlchemy)
UUID	Universally Unique Identifier, used as the ticket primary key
CORS	Cross-Origin Resource Sharing
Bedrock	Amazon Bedrock, the AWS managed foundation-model service used for AI summarization
CRUD	Create, Read, Update, Delete
DTO	Data Transfer Object, implemented here as Pydantic schemas

2. System Overview

2.1 High-Level Architecture

AI Service Desk is a three-tier application orchestrated with Docker Compose:

- Frontend – a React 19 / Vite single-page app (port 5173) that renders the ticket dashboard and calls the backend over HTTP via axios.
- Backend – a FastAPI application (port 8000) exposing REST endpoints for ticket CRUD and AI summarization, organized in API → service → repository layers.
- Database – PostgreSQL 17, accessed asynchronously through SQLAlchemy 2.0 and versioned with Alembic migrations.
- AI provider – Amazon Bedrock (Converse API), called from the backend to generate ticket summaries and suggested responses.

2.2 System flow - Key Components

Key backend components and their responsibilities:

Component	Responsibility
app/api	FastAPI routers (tickets.py, ai.py) – request/response handling and HTTP status mapping
app/services	Business logic (ticket_service.py) and the Bedrock AI client (services/aws)
app/repositories	Data-access layer that issues SQLAlchemy queries against the tickets table
app/models	SQLAlchemy ORM model for the Ticket entity
app/schemas	Pydantic request/response schemas and validation rules
app/core	Settings (config.py), dependency injection (deps.py), and custom exceptions
app/db	Async engine/session factory and the declarative Base
app/middleware	Request profiler that times each request and logs a cProfile breakdown

2.3 User Roles and Access Levels

The application does not currently implement authentication, authorization, or user accounts. Every endpoint is unauthenticated and reachable by any client that can reach the backend, so there is a single implicit role with full access to all ticket operations. Introducing role-based access control (e.g., agent vs. administrator) is a natural next step and is called out again in section 4.2 (Security).

3. Functional Requirements

3.1 Module/Feature Name

The system has two functional modules:

- Ticket Management (app/api/tickets.py) – create, list (with optional status/priority filters), retrieve, update, and delete support tickets.
- AI Summarization (app/api/ai.py) – accepts a free-text ticket description and returns an AI-generated summary and a suggested agent response, using AWS Bedrock.

3.2 Inputs/Outputs

Inputs and outputs per endpoint:

Endpoint	Input	Output
POST /tickets	title (str, 3–200 chars), priority (low/medium/high)	TicketResponse (id, title, priority, status, created_at, is_resolved)
GET /tickets	Optional query params: status, priority	List of TicketResponse
GET /tickets/{id}	ticket_id (UUID, path)	TicketResponse or 404
PATCH /tickets/{id}	Any subset of title, priority, status, assignee	Updated TicketResponse
DELETE /tickets/{id}	ticket_id (UUID, path)	Confirmation message
POST /ai/summarize	ticket_description (str, 10– 5,000 chars)	summary and suggested_response (strings)

3.3 Business Rules

Validation and business rules enforced by the schemas and service layer:

- Ticket titles must be 3–200 characters after trimming whitespace; blank titles are rejected.
- priority is restricted to low, medium, or high and defaults to medium.
- status is restricted to open, in_progress, resolved, or closed and defaults to open.
- A closed ticket cannot be reopened: TicketService.update_ticket raises a ValueError if a PATCH tries to move a closed ticket to any status other than closed.
- ticket_description for AI summarization must be 10–5,000 characters.

3.4 Error Handling

Error handling is centralized through typed exceptions and FastAPI handlers:

Condition	Exception	HTTP response
-----------	-----------	---------------

Ticket does not exist	<code>TicketNotFoundError</code>	404 with <code>{"error": "ticket_not_found", "id": ...}</code>
Invalid status transition (reopening a closed ticket)	<code>ValueError</code>	400 with the error message as detail
Bedrock call fails or returns malformed output	<code>BedrockServiceError</code>	502 “The AI service is temporarily unavailable”
Request validation failure (e.g., bad title length)	<code>pydantic ValidationError</code>	422 with field-level error detail (FastAPI default)

4. Non-Functional Requirements

4.1 Performance

A request-timing middleware (`app/middleware/profiler.py`) attaches an `X-Response-Time` header to every response and logs a `cProfile` breakdown of application-level function calls for each request, which can be used to spot regressions during development. A Locust script (`locust.py`) is included in the repository for load testing the ticket endpoints; no fixed throughput or latency targets are currently defined.

4.2 Security

There is currently no authentication or authorization on any endpoint, and CORS is configured to allow all origins, methods, and headers. Secrets (database credentials, AWS keys, `SECRET_KEY`) are supplied via environment variables / a `.env` file rather than hard-coded, but the `dockercompose.yml` file does contain a plaintext development database password, which should not be reused in any shared or production environment. Adding authentication and narrowing CORS are recommended before internet-facing deployment.

4.3 Availability

The backend exposes `GET /health` (verifies DB connectivity with `SELECT 1`) and `GET /ready` for use by orchestrators and load balancers. The Docker image also defines a native `HEALTHCHECK` that polls `/health` every 30 seconds. In `docker-compose.yml` the backend container depends on the Postgres container being healthy before it starts. No multi-instance failover or database replication is configured in this repository.

4.4 Maintainability

The codebase follows a layered architecture (`API` → `service` → `repository` → `model`), which keeps HTTP concerns, business rules, and persistence logic separate and independently testable. Schema changes are tracked with Alembic migrations, dependencies are pinned with a `uv` lock file, configuration is centralized and typed via `pydantic-settings`, and unit and integration tests (`pytest`) cover validation logic and the ticket CRUD flow.

5. Data Model / Database Design

5.1 Entity-Relationship Diagram

The current schema has a single entity, tickets, and no foreign-key relationships to other tables. As the AI Service Desk grows (e.g., adding users, agents, or ticket comments) an ER diagram will become useful; today the "diagram" is effectively the one table described below.

5.2 Tables and Fields

The tickets table (app/models/ticket.py) has the following columns:

Column	Type	Constraints	Notes
id	UUID	Primary key	Generated with <code>uuid.uuid4()</code> in Python
title	VARCHAR(200)	NOT NULL	Validated to be 3–200 chars, whitespace-trimmed
priority	ENUM(priority_enum)	low / medium / high, default medium	
status	ENUM(status_enum)	open / in_progress / resolved / closed, default open	
assignee	VARCHAR	Nullable	Free-text assignee name
assignee_email	VARCHAR(254)	Nullable	Not validated as an email format in the schema
created_at	TIMESTAMPTZ	Server default <code>now()</code>	Set by the database on insert

5.3 Indexes and Keys

id is the primary key (UUID). There are no additional explicit indexes defined on status or priority even though both are used as query filters in GET /tickets; adding indexes on these columns would be a reasonable optimization as the ticket volume grows. There are no foreign keys, since tickets is currently the only table.

5.4 Sample Data

Example ticket record returned by the API:

```
{  
  "id": "3fae2b7e-...",
```

```
"title": "Login page returns 500 error",  
"priority": "high", "status": "open",  
"created_at": "2026-07-24T09:15:00Z",  
"is_resolved": false  
}
```

6. Integration Points

6.1 External Systems

The application integrates with:

- Amazon Bedrock – foundation-model runtime used to summarize ticket descriptions and draft suggested responses (app/services/aws/bedrock_service.py).
- PostgreSQL 17 – the system of record for tickets, run as its own container in dockercompose.yml.

6.2 APIs Used

APIs exposed by the backend (all under the FastAPI app):

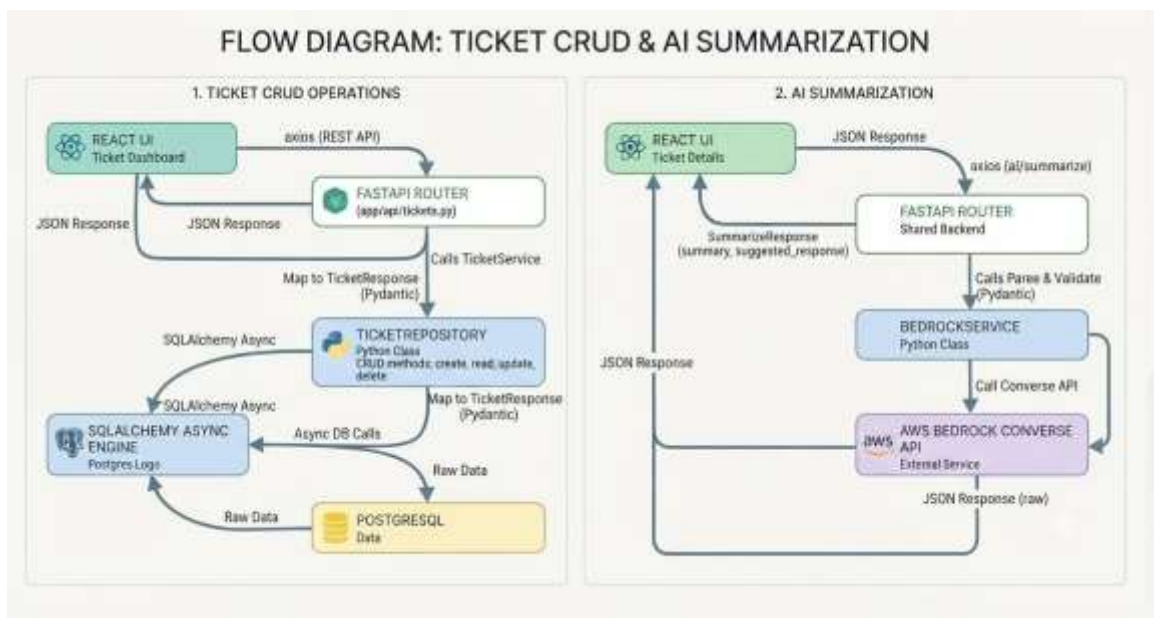
Method	Path	Purpose
GET	/	Welcome/root message
GET	/health	Liveness/DB-connectivity check
GET	/ready	Readiness check
POST	/tickets	Create a ticket
GET	/tickets	List tickets, optionally filtered by status/priority
GET	/tickets/{ticket_id}	Retrieve a single ticket
PATCH	/tickets/{ticket_id}	Partially update a ticket
DELETE	/tickets/{ticket_id}	Delete a ticket
POST	/ai/summarize	Summarize a ticket description via Bedrock

- The frontend consumes these endpoints through the axios wrapper in frontend/src/api/.

6.3 Data Flow Diagram

Two primary data flows:

- Ticket CRUD: React UI → axios → FastAPI router (app/api/tickets.py) → TicketService → TicketRepository → SQLAlchemy async engine → PostgreSQL, and the response is mapped back to a TicketResponse and returned as JSON.
- AI summarization: React UI → axios → /ai/summarize → BedrockService → AWS Bedrock Converse API → the JSON response is parsed and validated → returned as a SummarizeResponse (summary, suggested_response).



6.4 Authentication/Authorization

No end-user login, OAuth flow, or role-based access control exists in the current codebase; ticket endpoints are open. The only credentials in play are the backend's AWS access key/secret, used server-side to authenticate to Bedrock – this is service-to-service authentication, not user authentication.

7. Test Strategy

The testing strategy is designed to ensure high reliability, robust input validation, secure business logic enforcement, and reliable performance across the application lifecycle. The test suite is organized into three primary layers:

7.1 Test Scenarios

Unit Tests (tests/unit_test/test_unit.py)

Focus: Pure Python logic isolation without external dependencies or database overhead.

Scenarios Covered:

- Pydantic schema validation for TicketCreate and TicketUpdate.

- Boundary testing for title lengths (rejecting strings under 3 characters or over 200 characters, and blank spaces).
- Automated string sanitization (verifying whitespace stripping).
- Validation of permitted priority enums (low, medium, high) versus unauthorized values (urgent, critical, etc.) using parameterized tests.
- Computed property validation (is_resolved correctly evaluating to True exclusively for resolved status).
- Custom exception integrity (verifying TicketNotFoundError binds error IDs correctly).

Integration Tests (tests/integration_test/)

Focus: End-to-end HTTP request/response flows exercising FastAPI routers, business logic constraints, and SQLite persistence (sqlite+aiosqlite:///memory:).

Scenarios Covered:

- test_create.py (Creation Flows): Happy paths for ticket creation, default attribute assignments (status="open", is_resolved=False), whitespace handling, and handling invalid client payloads resulting in 422 Unprocessable Entity responses.
- test_get.py (Retrieval & Filtering Flows): Successful single-ticket fetching, handling nonexistent UUIDs (404 Not Found with custom error payloads), malformed ID rejection (422), multi-item listing, and complex query parameter filtering (by status and priority).
- test_update.py (Modification & State Transitions): Multi-field updates, chronological status transition lifecycles (open $\rightarrow$ in_progress $\rightarrow$ resolved), strict business rule enforcement (preventing closed tickets from being reopened with a 400 Bad Request), and missing record error handling.
- test_delete.py (Deletion Flows): Successful resource removal, verification of subsequent resource unreachability (404), and safe handling of non-existent deletions.

Load Testing (locust.py)

Focus: Performance, concurrency, and stress resilience.

Scenarios Covered:

Located at the repository root, a Locust script is provided to simulate concurrent virtual user traffic against core ticket endpoints under high-load conditions.

7.2 Expected Results

Valid inputs are expected to produce the documented HTTP status and response schema (e.g., 200/201 with a TicketResponse body); invalid inputs are expected to raise a pydantic ValidationError (422) or the domain-specific errors described in section 3.4 (404 for missing tickets, 400 for illegal status transitions, 502 for Bedrock failures).

7.3 Traceability to Requirements

test_create_ticket_invalid_priority / valid_priority and the title-length tests trace to the business rules in section 3.3. test_ticket_response_is_resolved traces to the TicketResponse.is_resolved

computed field in section 5.2. The integration tests under `tests/integration_test/` trace to the CRUD endpoints listed in section 6.2.

8. Deployment & Configuration

8.1 Environments

The repository defines a local development setup only – there is no separate QA, UAT, or production configuration checked in. Two ways to run it locally:

- Bare-metal dev: run the backend with uvicorn against a locally running PostgreSQL instance, and the frontend with the Vite dev server.
- Docker Compose: `docker-compose.yml` starts three containers – postgres, backend, and frontend – wired together with a shared network and environment variables.

8.2 Setup Steps

Local (non-Docker) setup:

- Backend: create/activate the virtual environment, then run `.venv/bin/uvicorn app.main:app --reload --port 8000` (dependencies installed via uv, see `pyproject.toml` / `uv.lock`).
- Database: `docker compose up -d postgres`, then `alembic upgrade head` to apply migrations.
- Frontend: `cd frontend && npm install && npm run dev` (Vite serves on port 5173).

8.3 Configuration Files and Parameters

Backend settings are declared as required fields in `app/core/config.py` (pydantic-settings) and loaded from a `.env` file:

Variable	Purpose
APP_NAME	Application name
API_VERSION	API version string
DEBUG	Debug mode toggle
DATABASE_URL	Async SQLAlchemy connection string (<code>postgresql+asyncpg://...</code>)
SECRET_KEY	Application secret key
ACCESS_TOKEN_EXPIRE_MINUTES	Reserved for future token-based auth
AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY	Credentials used to call Bedrock
AWS_REGION	AWS region for the Bedrock client

AWS_DEMO_MODE	Optional flag to run without live Bedrock calls
BEDROCK_MODEL_ID	Bedrock model identifier used for summarization
DATABASE_READY	Internal readiness flag

- docker-compose.yml additionally sets DATABASE_URL for the backend container to point at the postgres service, and the frontend reads VITE_API_URL at build/run time to know where the backend is hosted.

9. Appendix

Screenshots

1. Health Endpoint

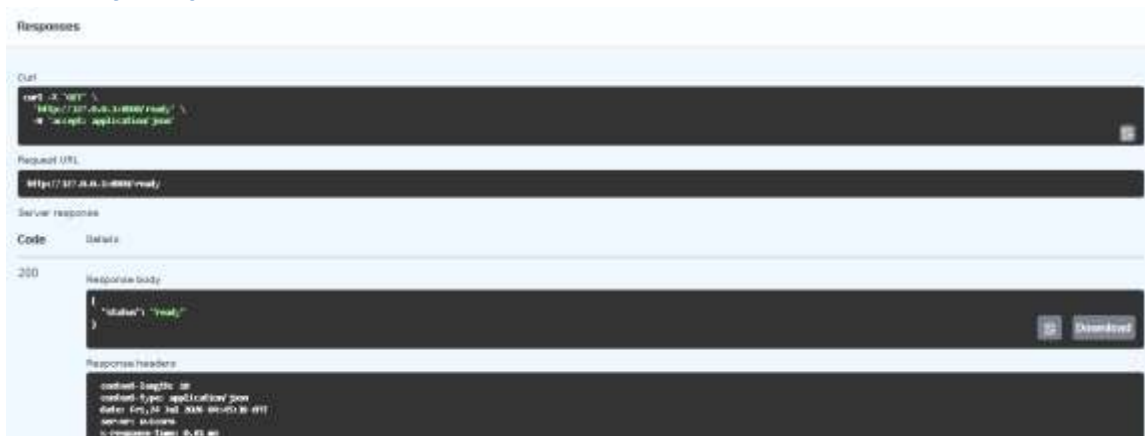


The screenshot shows a web browser interface with the following details:

- Out:** A code editor showing a curl command: `curl -s "http://127.0.0.1:8080/health/" -H "accept: application/json"`
- Request URL:** `http://127.0.0.1:8080/health`
- Server response:**
 - Code:** 200
 - Response body:** `{ "status": "healthy" }`
 - Response headers:**

```
content-length: 20
content-type: application/json
date: Sat, 04 Jul 2020 04:00:30 UTC
server: nginx
x-response-time: 0.01 ms
```

2. Ready Endpoint



The screenshot shows a web browser interface with the following details:

- Out:** A code editor showing a curl command: `curl -s "http://127.0.0.1:8080/ready/" -H "accept: application/json"`
- Request URL:** `http://127.0.0.1:8080/ready`
- Server response:**
 - Code:** 200
 - Response body:** `{ "status": "ready" }`
 - Response headers:**

```
content-length: 20
content-type: application/json
date: Fri, 03 Jul 2020 04:00:30 UTC
server: nginx
x-response-time: 0.01 ms
```

3. Ai Summary Endpoint

Responses

Curl

```
curl -s 'http://127.0.0.1:8000/ai/summary/' \
  -H 'accept: application/json' \
  -H 'content-type: application/json' \
  -d '{
    "ticket_description": "login failed"
  }'
```

Request URL

http://127.0.0.1:8000/ai/summary/

Server response

Code **Details**

200

Response body

```
{
  "message": "User is experiencing a login failure and is unable to access their account.",
  "response_message": "Thank you for reaching out. We're sorry to hear you're having trouble logging in. Please try resetting your password using the 'Forgot Password' link on the login page. If the issue persists, kindly provide your username or registered email address so we can investigate further and assist you promptly."
}
```

Response headers

```
access-control-allow-origin: *
access-control-expose-headers: *
content-length: 436
content-type: application/json
date: Fri, 04 Jul 2020 04:13:44 GMT
server: gunicorn
x-response-time: 200.43 ms
```

4. Delete Endpoint

Responses

Curl

```
curl -s 'http://127.0.0.1:8000/tickets/status?id=5456&token=6409021216' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8000/tickets/status?id=5456&token=6409021216

Server response

Code **Details**

200

Response body

```
{
  "message": "Ticket deleted successfully"
}
```

Response headers

```
access-control-allow-origin: *
access-control-expose-headers: *
content-length: 41
content-type: application/json
date: Fri, 04 Jul 2020 04:13:44 GMT
server: gunicorn
x-response-time: 10.07 ms
```


7. Post Endpoint

curl -X "POST" \
 "http://172.16.1.1000/tickets" \
 -H "accept: application/json" \
 -H "Content-Type: application/json" \
 -d '{
 "title": "bug called",
 "priority": "high"
 }'

Request URL:
http://172.16.1.1000/tickets

Server response:

Code Details

200

Response body

```
{
  "id": "5000000-1000-0000-0000-000000000000",
  "title": "bug called",
  "priority": "high",
  "status": "open",
  "created_at": "2020-07-20T10:00:00.000Z",
  "is_resolved": false
}
```

Response headers

```
access-control-allow-origin: *
access-control-expose-headers: *
content-length: 200
content-type: application/json
date: Fri, 20 Jul 2020 09:59:47 GMT
server: akka
x-response-time: 80.30 ms
```

8. Get Endpoint

Responses

curl -X "GET" \
 "http://172.16.1.1000/tickets" \
 -H "accept: application/json"

Request URL:
http://172.16.1.1000/tickets

Server response:

Code Details

200

Response body

```
[
  {
    "id": "5000000-1000-0000-0000-000000000000",
    "title": "Ticketing",
    "priority": "medium",
    "status": "open",
    "created_at": "2020-06-24T10:00:00.000Z",
    "is_resolved": false
  },
  {
    "id": "5000000-1000-0000-0000-000000000000",
    "title": "Ticketing",
    "priority": "medium",
    "status": "open",
    "created_at": "2020-07-03T10:00:00.000Z",
    "is_resolved": false
  },
  {
    "id": "5000000-1000-0000-0000-000000000000",
    "title": "Ticket",
    "priority": "medium",
    "status": "open",
    "created_at": "2020-06-25T07:50:00.000Z",
    "is_resolved": false
  },
  {
    "id": "5000000-1000-0000-0000-000000000000",
    "title": "Ticketing",
    "priority": "medium",
    "status": "open",
    "created_at": "2020-06-25T07:50:00.000Z",
    "is_resolved": false
  }
]
```

Response headers

```
content-length: 369
content-type: application/json
date: Fri, 20 Jul 2020 09:59:47 GMT
server: akka
x-response-time: 86.33 ms
```

9. Ai Component

AI Assistant

login failed

generate summary

Summary

User is experiencing a login failure and is unable to access their account.

Suggested Response

Thank you for reaching out. We're sorry to hear you're having trouble logging in. Please try resetting your password using the 'Forgot Password' link on the login page. If the issue persists, kindly provide any error messages you're seeing so we can investigate further and assist you promptly.

10. Specific Ticket Detail

Ticket Details
Title: License key issue
Priority: low
Status: closed
Created At:

11. Tickets

Tickets

updated
Priority: medium
Status: open

View Edit Delete

strisdndng
Priority: medium
Status: open

View Edit Delete

dstsdldfs
Priority: medium
Status: open

View Edit Delete

12. Create Ticket

AI Service Desk

System Status
Health: healthy
Ready: ready
Response Time: 96.22 ms

Create Ticket
title

priority
Medium

13. Edit Ticket

Edit Ticket
License key issue

Low

Closed

References

FastAPI: <https://fastapi.tiangolo.com> — SQLAlchemy: <https://docs.sqlalchemy.org> — Alembic: <https://alembic.sqlalchemy.org> — Amazon Bedrock: <https://docs.aws.amazon.com/bedrock> — React: <https://react.dev> — Vite: <https://vite.dev>